

AI-Powered Zero Trust Network Intrusion Detection System

PHASE 1 COMPLETE STUDENT STUDY GUIDE & VIVA MASTERY MANUAL

Target Audience: Final-Year Computer Science & Cybersecurity Engineering Student

Purpose: Viva Preparation, Coordinator Review, & Technical Project Defense

Scope: Phase 1 Zero Trust Security, Preprocessing, LSTM Deep Learning & Evaluation

Academic Year: 2026

Chapter 1: Project Overview & Core Philosophy

1.1 What is a Network Intrusion Detection System (NIDS)?

Explanation: A Network Intrusion Detection System (NIDS) acts like a digital security camera for computer networks. It constantly monitors data packets passing through network cables or routers and checks whether any packet is a cyber attack (such as a hacker trying to crash a server or scan open ports).

Example: Imagine a bank teller window. A standard transaction is a normal customer depositing money. A NIDS flags an intruder who tries to slip counterfeit notes through the window.

Why We Use It: Without a NIDS, hackers can silently probe network ports, steal passwords, or launch Denial-of-Service attacks without anyone knowing until systems crash.

1.2 Why Artificial Intelligence (AI) is Used Instead of Old Rules

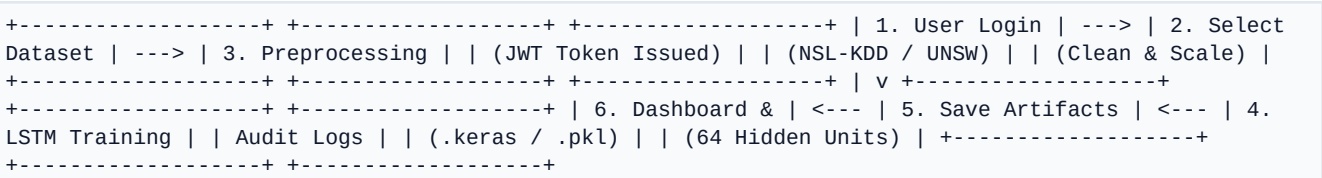
Explanation: Traditional NIDS systems relied on rigid IF-ELSE rules or static signatures (like an antivirus checking for exact past file names). However, hackers change their attack code daily. AI algorithms learn baseline normal patterns and can detect brand new, unseen cyber attacks (zero-day attacks).

1.3 What is Zero Trust Architecture?

Explanation: Traditional security followed the 'Perimeter Model'—like a castle with a moat. Once someone crossed the moat, they could enter any room. **Zero Trust** replaces this with the core rule: *'Never Trust, Always Verify'*.

Every request made to our web application must present a valid cryptographically signed JWT token proving who the user is and what their role permits.

1.4 Phase 1 Workflow Flowchart



Chapter 2: Project Architecture & Folder Breakdown

Every folder in our codebase has a single dedicated responsibility. Here is how they communicate:

File / Folder	What it Contains / Does	Why it Exists & Execution Context
app.py	Flask Application Factory & Route Registrar	Entry point of server. Starts app, initializes SQLite database, and registers blueprints.
config.py	Global Configuration Class (`Config`)	Stores SECRET_KEY, JWT expiration time, upload folder paths, and database URIs.
database/	SQLite Models (`init_db.py`)	Contains SQLAlchemy ORM classes: User, Role, DatasetHistory, ModelHistory, AuditLog.
security/	Zero Trust Security Modules	Includes `jwt_auth.py` (token generation), `password.py` (Bcrypt), and `zero_trust.py` (RBAC decorators).
preprocessing/	Data Preprocessing Pipeline	Cleans raw CSVs, label encodes text features, scales numbers, and shapes 3D LSTM tensors.

training/	Keras Model Trainer (`train_model.py`)	Runs background training thread, compiles LSTM network, logs progress, and saves `.keras` model.
api/	Flask REST Blueprints	Contains `auth_routes.py`, `dataset_routes.py`, and `model_routes.py` API endpoints.
models/	Phase 1 Serialized Output Directory	Stores final output artifacts: `nids_lstm_model.keras`, `scaler.pkl`, and `label_encoder.pkl`.
templates/	Jinja2 HTML Web Interfaces	Renders full-width top navigation views: `dashboard.html`, `dataset_list.html`, `train_model.html`, etc.

Chapter 3: Zero Trust Security & Authentication Mechanics

3.1 Password Hashing with Bcrypt

Explanation: We never store plain passwords like 'Password123' in the database. If a hacker steals the database file, plain passwords allow instant account hijacking. Instead, we use **Bcrypt salting and hashing**.

Code Snippet & Line-by-Line Breakdown:

```
salt = bcrypt.gensalt(rounds=12) hashed_password = bcrypt.hashpw(password.encode('utf-8'), salt)
```

- `salt = bcrypt.gensalt(rounds=12)`: Generates a random 16-byte salt string with a computational work factor of 12 rounds to defeat Rainbow Table attacks.
- `password.encode('utf-8')`: Converts string text into raw byte format required by cryptography algorithms.
- `bcrypt.hashpw(...)`: One-way mathematical transformation resulting in a 60-character hash stored safely in the database.

3.2 JWT Token Authentication & RBAC Decorator

Explanation: After a user logs in successfully, the server creates a digital wristband called a **JWT (JSON Web Token)**. Every REST API endpoint uses the `@require_role(['Admin'])` decorator to verify the token before allowing access.

Viva Tip — Why check JWT on every single API request?

Answer: In a Zero Trust model, there are no 'safe pages'. An attacker could attempt to bypass the frontend UI and send raw HTTP POST calls directly to '/api/dataset/delete/1'. The backend @jwt_required decorator intercepts every raw request and rejects unauthorized calls with a 401 Unauthorized or 403 Forbidden status code.

Chapter 4: Dataset Catalog & Ingestion Module

4.1 Supported Benchmark Datasets

Our Phase 1 system supports major benchmark datasets used in cybersecurity research:

- **NSL-KDD:** An improved benchmark dataset resolving duplicate records from the classic KDD'99 dataset. Contains features like duration, protocol_type, service, flag, src_bytes, dst_bytes.
- **UNSW-NB15:** Created by the Australian Centre for Cyber Security. Features modern synthetic attack behaviors (Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance).
- **CICIDS2017:** Contains modern real-world benign and attack traffic flows generated by the Canadian Institute for Cybersecurity.
- **Custom CSVs:** User-uploaded CSV datasets conforming to standard feature column headers.

4.2 Pre-Installed Test Datasets

To enable out-of-the-box testing without requiring manual file uploads, `init_db.py` automatically seeds 2 benchmark datasets into `datasets/raw/` on startup:

- 1. `nsl_kdd_benchmark_sample.csv` (1,000 samples; Pre-selected active dataset).
- 2. `unsw_nb15_benchmark_sample.csv` (1,000 samples).

Chapter 5: Data Preprocessing Pipeline

5.1 Step-by-Step Data Preprocessing Pipeline

Preprocessing Step	Why It Is Required	Transformation Example
1. Missing Value Imputation	Neural networks crash if fed NaN or empty values.	Fills missing numerical values using median statistics.
2. Duplicate Removal	Prevents model from memorizing identical repeated rows.	Drops duplicate rows via <code>df.drop_duplicates()</code> .
3. LabelEncoding	Computers cannot multiply string text like 'tcp' or 'udp'.	Converts text categories: ['tcp','udp','icmp'] -> [0, 1, 2].
4. StandardScaler Normalization	Large values (e.g. 100,000 bytes) overpower small values (e.g. 0.5s duration).	Transforms numbers to Mean = 0, StdDev = 1 using $z = (x - \mu) / \sigma$.
5. 3D Sequence Reshaping	LSTM memory layers require a 3D time-step tensor input.	Reshapes matrix from (samples, features) -> (samples, 1, features).

Chapter 6: Feature Engineering & Attribute Analysis

Here is how key network features help our LSTM AI engine detect cyber attacks:

Network Feature	Description	How it Helps Spot Attacks
<code>duration / dur</code>	Connection duration in seconds	Extremely short connections (<0.01s) repeated quickly indicate port scans; extremely long connections may indicate data exfiltration.
<code>src_bytes / sbytes</code>	Bytes sent from source host	Unusually large outgoing byte counts indicate data theft or outbound file transfer attacks.
<code>dst_bytes / dbytes</code>	Bytes received from destination	Massive incoming bytes indicate high-volume flood attacks (e.g. UDP/ICMP Floods).
<code>count / rate</code>	Connections to same host in window	High count values (>200 connections/sec) flag Denial-of-Service (DoS) flooding.
<code>flag / state</code>	TCP connection flags (SF, S0, REJ)	'S0' (SYN sent, no ACK received) flags SYN Flood attacks; 'REJ' flags rejected port probes.
<code>dst_port</code>	Target destination port number	Probing administrative ports like 22 (SSH) or 3389 (RDP) signals brute-force reconnaissance.

Chapter 7: LSTM Deep Learning Model (Taught From Scratch)

7.1 What is an LSTM Neural Network?

Explanation: A **Long Short-Term Memory (LSTM)** network is a special type of Recurrent Neural Network (RNN). Traditional neural networks treat every sample independently (they have no memory). LSTMs contain internal memory gates (Input Gate, Forget Gate, Output Gate) that remember historical network traffic patterns over time.

Viva Question — Why LSTM instead of Decision Trees or CNNs?

Answer: Decision Trees treat each packet as an isolated row, ignoring time context. CNNs are optimized for 2D spatial grid images. Network traffic is sequential time-series data. LSTMs maintain an internal memory state across sequential packets, making them superior for detecting multi-step attacks.

7.2 Model Architecture Code & Line-by-Line Breakdown

```
model = Sequential([ LSTM(64, input_shape=(1, X_train.shape[2])), return_sequences=False),
                    Dropout(0.2), Dense(32, activation='relu'), Dense(num_classes, activation='softmax' if num_classes > 2
                    else 'sigmoid') ]) model.compile(optimizer='adam', loss='categorical_crossentropy',
                    metrics=['accuracy'])
```

- `Sequential([...])`: A linear stack of neural network layers where data flows sequentially from top to bottom.
- `LSTM(64, input_shape=(1, features))`: 64 hidden LSTM units extracting temporal sequence patterns.
- `Dropout(0.2)`: Randomly deactivates 20% of neurons during each training step to prevent model overfitting (memorization).
- `Dense(32, activation='relu')`: Fully-connected layer applying Rectified Linear Unit (ReLU) activation to learn non-linear feature combinations.
- `Dense(num_classes, activation='softmax')`: Output layer producing class probabilities summing to 1.0 (100%).
- `model.compile(optimizer='adam', loss='categorical_crossentropy')`: Adam adaptive learning optimizer paired with Categorical Cross-Entropy loss function.

Chapter 8: Model Training Mechanics & Backpropagation

8.1 How Training Works

1. **Forward Pass:** Input features pass through the LSTM and Dense layers to produce initial predictions.
2. **Loss Computation:** Cross-entropy loss calculates the mathematical error between predicted probabilities and actual target labels.
3. **Backpropagation:** Gradients are calculated using calculus chain rules, pushing error signals backward through neural layers.
4. **Weight Update:** The **Adam optimizer** adjusts internal synaptic weights to reduce loss in subsequent epochs.

8.2 Epochs & Batch Size

- **Epoch (Default = 10):** One complete training pass through the entire dataset.
- **Batch Size (Default = 64):** Number of training samples processed together before updating weights.

Chapter 9: Model Evaluation & Performance Analytics

9.1 Mathematical Metrics

Metric Name	Mathematical Formula	Plain English Explanation
Accuracy	$(TP + TN) / Total$	Overall percentage of correctly classified normal and attack traffic.
Precision	$TP / (TP + FP)$	Out of all alerts flagged as attacks, what fraction were real attacks?
Recall (Sensitivity)	$TP / (TP + FN)$	Out of all actual attacks in the dataset, what fraction did our model catch?
F1-Score	$2 * (P * R) / (P + R)$	Harmonic mean balancing Precision and Recall.

Common Examiner Trap — When can high accuracy be misleading?

Answer: In imbalanced datasets where 99% of traffic is Normal and 1% is Attack, a naive model that predicts 'Normal' for every packet gets 99% accuracy! However, its Recall for attacks is 0%. That is why we evaluate Precision, Recall, F1-Score, and the Confusion Matrix alongside Accuracy.

Chapter 10: Executive SOC Dashboard Architecture

The full-width executive dashboard ([templates/dashboard.html](file:///c:/Users/hp/Desktop/ML/templates/dashboard.html)) presents 6 KPI Telemetry Cards:

- Uploaded Datasets:** Total count of registered CSV datasets in SQLite.
- Active Dataset:** Name of the dataset currently selected for training.
- Dataset Rows & Cols:** Sample row count and attribute column count.
- Model Status:** Shows 'Trained' or 'Not Trained'.
- Training Accuracy:** Final evaluation accuracy percentage.
- Last Training:** Timestamp of most recent training run.

Chapter 11: Database Schema & Relationships

Database Table	Key Columns / Attributes	Purpose & Relationships
users	id, username, password_hash, role_id	Stores user accounts. Foreign key to roles table.
roles	id, name ('Admin', 'User')	Defines RBAC permission levels.
dataset_history	id, filename, dataset_type, row_count, col_count, is_selected	Catalog of uploaded/seeded CSV datasets.
model_history	id, model_name, accuracy, loss, trained_at, filepath	Log of trained model runs and evaluation metrics.
audit_logs	id, action, status, username, ip_address, timestamp, details	Immutable security event log feed.

Chapter 12: End-to-End Execution Pipeline

- User Sign-In:** User logs in via /login. Server verifies password hash via Bcrypt and issues JWT token.

2. **Dataset Selection:** User views /dataset-list and selects an active dataset (e.g. nsl_kdd_benchmark_sample.csv).
3. **Preprocessing Trigger:** User navigates to /train-model and clicks 'Start Training Model'. Flask API launches background thread.
4. **Pipeline Execution:** Data is loaded, cleaned, LabelEncoded, StandardScaler normalized, and reshaped into 3D tensors.
5. **Keras LSTM Training:** 64-unit LSTM model trains across 10 epochs. Frontend polls /api/model/status every 1.5s to update Chart.js.
6. **Model Serialization:** Trained model saved to models/nids_lstm_model.keras; scaler saved to models/scaler.pkl.
7. **Evaluation & Audit Log:** Model evaluated on test set; metrics written to JSON report; event logged in audit trail.

Chapter 13: Code Walkthrough of Key Functions

13.1 Preprocessing Function (preprocessing/preprocess.py)

```
def preprocess_dataset(filepath): df = pd.read_csv(filepath).drop_duplicates() for col in
df.select_dtypes(include=['object']).columns: le = LabelEncoder() df[col] =
le.fit_transform(df[col].astype(str)) scaler = StandardScaler() X_scaled = scaler.fit_transform(X)
X_3d = X_scaled.reshape((X_scaled.shape[0], 1, X_scaled.shape[1])) return X_3d, y, scaler, le
```

- df.drop_duplicates(): Removes identical rows.
- LabelEncoder().fit_transform(...): Converts categorical strings to integers.
- StandardScaler().fit_transform(X): Normalizes features to mean=0, stddev=1.
- X_scaled.reshape((samples, 1, features)): Converts 2D matrix into 3D LSTM tensor format.

Chapter 14: 50 Coordinator Review & Viva Questions (With Simple Answers)

1. What is an NIDS?

A system that monitors network traffic to detect unauthorized intrusion and attack patterns.

2. What is Zero Trust?

A security framework operating on 'Never Trust, Always Verify', checking identity and permissions on every request.

3. Why use JWT instead of sessions?

JWT is stateless, scalable, and stores signed user identity in tokens without burdening server memory.

4. What algorithm is used for password hashing?

Bcrypt with a 12-round computational salt.

5. What is RBAC?

Role-Based Access Control, restricting administrative actions (e.g. train/delete) to Admin roles.

6. Why use LSTM for NIDS?

Network traffic is sequential time-series data. LSTMs maintain internal memory to detect temporal attack sequences.

7. Why not Decision Trees?

Decision Trees process each packet independently without temporal memory across time steps.

8. Why not CNNs?

CNNs excel at 2D grid image feature extraction, whereas LSTMs excel at 1D sequential data.

9. What is the role of Flask in this project?

Flask acts as the lightweight Python web framework hosting REST APIs and serving UI views.

10. What database is used?

SQLite managed via Flask-SQLAlchemy ORM.

11. What is LabelEncoding?

Converting categorical text values (e.g. 'tcp') into integers (e.g. 0).

12. What is StandardScaler?

Normalizing feature distributions to Mean = 0 and Standard Deviation = 1.

13. Why do we scale features?

To prevent large numerical values (e.g. 100,000 bytes) from dominating model weight updates.

14. Why reshape data to 3D?

Keras LSTM layers require input shape `(batch\_size, time\_steps, features)`.

15. What is an Epoch?

One complete forward and backward pass through the entire training dataset.

16. What is Batch Size?

Number of samples processed before updating internal neural network weights.

17. What optimizer is used?

Adam (Adaptive Moment Estimation), combining AdaGrad and RMSProp for fast convergence.

18. What loss function is used?

Categorical Cross-Entropy for multi-class classification (or Binary Cross-Entropy for 2 classes).

19. What is Dropout?

Randomly deactivating a fraction of neurons (20%) during training to prevent overfitting.

20. What is Dense Layer?

A standard fully-connected neural network layer.

21. What is Softmax?

An activation function converting output logits into class probabilities summing to 1.0.

22. What is ReLU?

Rectified Linear Unit ($\max(0, x)$), providing fast non-linear activation.

23. What is Accuracy?

Total correct predictions divided by total predictions.

24. Why can high accuracy be misleading?

In imbalanced datasets, predicting only the majority class gives high accuracy but fails to catch attacks.

25. What is Precision?

Fraction of flagged attacks that were actual attacks ($TP / (TP + FP)$).

26. What is Recall?

Fraction of actual attacks successfully detected ($\text{TP} / (\text{TP} + \text{FN})$).

27. What is F1-Score?

Harmonic mean of Precision and Recall ($2 * P * R / (P + R)$).

28. What is a Confusion Matrix?

A table comparing actual vs predicted classes showing TP, FP, TN, FN.

29. What is ROC Curve?

Receiver Operating Characteristic curve plotting True Positive Rate vs False Positive Rate.

30. What is AUC?

Area Under the ROC Curve; higher AUC (near 1.0) indicates better class separation.

31. What datasets are pre-installed?

NSL-KDD and UNSW-NB15 sample datasets (1,000 rows each).

32. Where are trained models saved?

In `models/nids_lstm_model.keras`.

33. Where are scalers saved?

In `models/scaler.pkl` using Python pickle serialization.

34. Where are evaluation metrics stored?

In `reports/latest_evaluation_report.json`.

35. What is the Audit Log?

An immutable database table tracking user actions, logins, dataset uploads, and training runs.

36. Can audit logs be exported?

Yes, via one-click CSV export on the dashboard.

37. What Python version is used?

Python 3.11+.

38. What deep learning framework is used?

TensorFlow 2.x with Keras API.

39. What is `app.py`?

Application Factory initializing Flask, database, and blueprints.

40. What is `config.py`?

Central configuration file storing secret keys, folder paths, and parameters.

41. How does frontend communicate with backend?

Via asynchronous `fetch()` API calls sending JSON payloads and JWT headers.

42. What frontend framework is used?

Bootstrap 5 for responsive CSS layout.

43. What charting library is used?

Chart.js for rendering live loss/accuracy graphs.

44. How is training progress monitored live?

Frontend polls `/api/model/status` every 1.5s to fetch current epoch metrics.

45. What is overfitting?

When a model memorizes training data noise and performs poorly on unseen test data.

46. How do we prevent overfitting?

Using 20% Dropout layers and Train/Test dataset splitting.

47. What is the default Train/Test split ratio?

80% Training, 20% Testing.

48. What is `is\_selected` in DatasetHistory?

A boolean flag indicating which dataset is currently active for training.

49. Can common users delete datasets?

No, dataset deletion requires Admin role protected by `@require\_role(['Admin'])`.

50. What is the ultimate deliverable of Phase 1?

A serialized `.keras` LSTM model, `.pkl` scaler, `.pkl` label encoder, and metric evaluation report.

Chapter 15: Phase 2 Integration Preview

Explanation: Phase 1 ends cleanly after the AI model (`nids\_lstm\_model.keras`) and scaler (`scaler.pkl`) are trained, evaluated, and serialized. In future Phase 2 development, a live packet sniffer engine (e.g. Scapy or raw sockets) can be added as a separate module. It will capture network traffic frames, extract identical features, scale them using `scaler.pkl`, and pass them into `nids\_lstm\_model.keras` for real-time attack detection without needing to alter any Phase 1 code!

Final Study Summary

Phase 1 provides the complete foundation: Zero Trust Security + Data Preprocessing + LSTM AI Model Training + Evaluation Metrics + SOC Dashboard. You are fully equipped to explain and defend every line of code!